Universidade Aberta
Licenciatura em Engenharia Informática
Unidade Curricular 21184 — Projeto de Engenharia Informática
Ano Lectivo 2025/2026

ForensiQ

Plataforma Modular de Gestão de Prova Digital
para *First Responders*

Relatório Intercalar (Fase 2)

Autor: João Rodrigues
N.º de aluno: 2203474
Orientador: Prof. Pedro Duarte Pestana
Data: 6 de Maio de 2026

Resumo

O ForensiQ é uma plataforma *web mobile-first* que digitaliza o registo, a preservação e a transferência de prova digital pelos agentes que primeiro chegam a uma cena de crime. O problema endereçado é a fragilidade da cadeia de custódia em ocorrências policiais com componente digital, hoje gerida por meios manuais (papel, fotografias no telemóvel pessoal, anotações dispersas). A solução estrutura-se em três pilares: integridade verificável por *hash* SHA-256 encadeado, conforme ISO/IEC 27037:2012; autorização granular por papéis (agente *vs.* perito) com auditoria transversal; e demonstração assíncrona em produção em forensiq.pt, que dispensa configuração local pelo avaliador.

Este relatório intercalar (Fase 2 da UC 21184) cobre os Capítulos 1 (Introdução), 2 (Desenho) e o estado actual da implementação descrito no início do Capítulo 3, conforme o Guia de Projecto do orientador. À data da entrega, o MVP está implementado e em produção, com **213 testes automatizados a passar**, conformidade demonstrada com a norma ISO/IEC 27037 e auditoria externa de postura de segurança (Qualys SSL Labs A+, Mozilla HTTP Observatory A+, submissão à lista HSTS preload). As lacunas conhecidas — nomeadamente as alíneas c) e d) do RGPD Art. 32 e a dependência de controlo organizacional sobre o acesso DBA — estão honestamente documentadas e enquadradas como trabalho futuro.

Palavras-chave: cadeia de custódia digital; ISO/IEC 27037; SHA-256; prova digital; Django; *first responder*; RGPD.

Índice

Lista de Acrónimos **5**

1 Introdução **7**
1.1 Descrição do projecto . . . 7
1.2 Âmbito e enquadramento . . . 7
1.3 Necessidades identificadas . . . 8
1.4 Objectivos . . . 8
1.5 Restrições . . . 9
1.6 Resultados esperados . . . 9
1.7 Calendário planeado . . . 10

2 Desenho **11**
2.1 Visão geral da solução . . . 11
2.2 Arquitectura . . . 11
2.2.1 Diagrama C4 nível 1 — Contexto . . . 11
2.2.2 Diagrama C4 nível 2 — Contentores . . . 11
2.2.3 Stack tecnológica . . . 12
2.2.4 Modelo de dados . . . 13
2.2.5 Decisões de arquitectura . . . 16
2.3 Algoritmo de certificação e integridade . . . 17
2.3.1 Modelo de ameaças aplicado . . . 17
2.3.2 Escolha do algoritmo de *hash* . . . 18
2.3.3 Hash de integridade do registo de evidência . . . 18
2.3.4 Cadeia de custódia com *hash* encadeado . . . 19
2.3.5 Concorrência e atomicidade . . . 20
2.3.6 Imutabilidade em três camadas . . . 21
2.4 Segurança transversal . . . 22
2.4.1 Autenticação JWT em *cookies* HttpOnly . . . 22
2.4.2 Autorização baseada em papéis (RBAC) . . . 23
2.4.3 Cabeçalhos HTTP de segurança . . . 23
2.4.4 Outras defesas . . . 24
2.5 Conformidade ISO/IEC 27037 e RGPD Art. 32 . . . 24
2.5.1 Mapeamento à norma ISO/IEC 27037:2012 . . . 24
2.5.2 Conformidade com o RGPD Art. 32 . . . 25
2.5.3 Evidência externa de postura de segurança . . . 26
2.6 Interface do utilizador . . . 26
2.6.1 Princípios de desenho . . . 26
2.6.2 Vistas-chave . . . 27
2.6.3 Wizard de evidência . . . 27
2.6.4 Máquina de estados de custódia . . . 27

3 Implementação — Estado actual **28**

3.1 Decomposição do trabalho 28
3.2 Estado final por componente 29
3.3 Cumprimento dos critérios de aceitação 30
3.4 Desvios ao plano 31
3.5 Demonstração em produção 32
3.6 Estado dos testes automatizados 33
3.7 Calendário actualizado para a fase restante 33

Uso de IA generativa **34**

Lista de Figuras

1 Diagrama C4 nível 1: ForensiQ, utilizadores e sistemas externos. 12
2 Diagrama C4 nível 2 (contentores) com tecnologias principais. 14
3 Modelo entidade-relação do ForensiQ. 15
4 Fluxograma do cálculo do *hash* encadeado e da detecção de adulteração. . . . 19
5 Defesa em profundidade contra adulteração: ORM, DRF e *triggers* PostgreSQL. 21
6 Máquina de estados da cadeia de custódia. Transições válidas a partir de cada estado; `DEVOLVIDA` e `DESTRUIDA` são terminais. 27

Lista de Tabelas

1 Objectivos específicos F1–F6 do MVP e respectivos critérios de aceitação observáveis. 9
2 Calendário das 16 semanas com marcos formais sublinhados. 11
3 Pilha tecnológica do ForensiQ com justificação por camada. 13
4 Síntese dos dez ADRs em vigor; detalhe em `docs/architecture/adr/`. . . . 17
5 Mapeamento sintético da ISO/IEC 27037:2012 aos mecanismos do ForensiQ. 25
6 Sumário executivo da conformidade RGPD Art. 32. 26
7 Resultado dos três testes externos de segurança (18 Abr 2026). 26
8 Decomposição de tarefas com precedências. 29
9 Estado por componente à data do intercalar. 30
10 Cumprimento dos requisitos funcionais e não-funcionais. 31
11 Calendário actualizado para a fase restante. 33

Lista de Acrónimos

ADR *Architecture Decision Record*
API *Application Programming Interface*
CIA-R Confidentiality, Integrity, Availability, Resilience
CSP *Content Security Policy*
CSRF *Cross-Site Request Forgery*
DAST *Dynamic Application Security Testing*
DRF Django REST Framework
ENFSI *European Network of Forensic Science Institutes*
GPS *Global Positioning System*
HSTS *HTTP Strict Transport Security*
IDOR *Insecure Direct Object Reference*
IMEI *International Mobile Equipment Identity*
IMSI *International Mobile Subscriber Identity*
ISO/IEC *International Organization for Standardization / International Electrotechnical Commission*
JWT *JSON Web Token*
MAC *Media Access Control*
MIME *Multipurpose Internet Mail Extensions*
MVP *Minimum Viable Product*
NIST *National Institute of Standards and Technology*
NTP *Network Time Protocol*
NUIPC Número Único de Identificação Processual de Crime
OCSP *Online Certificate Status Protocol*
OWASP *Open Web Application Security Project*
PII *Personally Identifiable Information*
PSP Polícia de Segurança Pública
RBAC *Role-Based Access Control*
RF Requisito Funcional
RGPD Regulamento Geral sobre a Protecção de Dados (UE 2016/679)
RNF Requisito Não-Funcional
SAST *Static Application Security Testing*
SCA *Software Composition Analysis*
SHA *Secure Hash Algorithm*
SRI *Subresource Integrity*
STRIDE *Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*
TLS *Transport Layer Security*

TSA *Time-Stamping Authority* (RFC 3161)

UAb Universidade Aberta

UC Unidade Curricular

VIN *Vehicle Identification Number*

WCAG *Web Content Accessibility Guidelines*

WSGI *Web Server Gateway Interface*

1 Introdução

O ForensiQ é uma aplicação *web mobile-first* que digitaliza e padroniza o registo, a preservação e a transferência de prova digital pelos agentes que primeiro chegam a uma cena de crime. Este relatório intercalar apresenta o estado do projecto à data de 6 de Maio de 2026, em cumprimento do calendário da Unidade Curricular 21184 da Universidade Aberta. O documento está estruturado em três capítulos, conforme o Guia de Projecto do orientador: introdução (presente capítulo), desenho (Capítulo 2) e implementação no estado actual (Capítulo 3).

1.1 Descrição do projecto

Os agentes da Polícia de Segurança Pública (PSP) que atuam como *first responders* em ocorrências com componente digital recorrem hoje a métodos manuais para registar e preservar a prova: papel, fotografias avulsas no telemóvel pessoal, anotações manuscritas e etiquetas em saquetas. O processo é vulnerável em várias dimensões. A cadeia de custódia torna-se frágil: não existe registo verificável de quem manuseou a prova, em que momento e em que condições. A prova carece frequentemente de localização exacta da apreensão e de *timestamp* sincronizado. As práticas variam entre agentes e entre divisões, o que dificulta auditoria processual posterior. Em tribunal, fragilidades de cadeia de custódia podem inviabilizar prova relevante, com consequências para a justiça material (Casey, 2011; Association of Chief Police Officers, 2012).

O autor identificou o problema a partir da experiência directa enquanto agente da PSP e perito digital forense. O tema foi formalizado como auto-proposta na fase preliminar de selecção do tema (anterior ao *kick-off* de Fase 1), em alinhamento com o Projeto #38 (Pestana, 2025) da licenciatura, com a delimitação explícita de que o ForensiQ não realiza aquisição de prova — apenas a sua *gestão* e *controlo* após apreensão.

A solução é uma plataforma *web* servida pelo *backend* Django, acedida por *browser* a partir de *smartphone* ou computador, sem necessidade de instalação de *software* especializado. Os registos são imutáveis por desenho, com integridade verificável por terceiros através de *hash* SHA-256, e a cadeia de custódia é encadeada criptograficamente para que qualquer adulteração posterior seja detectável.

1.2 Âmbito e enquadramento

O âmbito do ForensiQ cobre o ciclo completo *registo* → *preservação* → *transferência* da prova após apreensão. A versão actual é *digital-only*, conforme registado no ADR-0010, com a estrutura preparada para módulos forenses adicionais (biológico, químico, físico) em trabalho futuro. Estão fora do âmbito, de forma explícita: a aquisição forense de imagens de discos e telemóveis (*carving*, `dd`, FTK Imager); a análise de *malware*; a peritagem de ADN ou AFIS; a integração com sistemas internos da PSP, que requereria autorizações não disponíveis em contexto académico; e o uso de dados reais de processos judiciais, por questões legais e de confidencialidade.

1.3 Necessidades identificadas

A análise do contexto operacional, validada com o orientador na proposta de Fase 1, identificou três necessidades transversais que estruturam todo o desenho da solução:

1. **Integridade verificável.** Qualquer alteração posterior a um registo de prova tem de ser detectável por um terceiro que tenha acesso ao identificador e aos campos públicos. Esta necessidade liga-se directamente ao §5.4 da norma ISO/IEC 27037:2012 (ISO/IEC, 2012).
2. **Rastreabilidade auditável.** Cada manuseamento da prova — quem, quando, em que estado — tem de ficar registado de forma imutável, com sequência canónica que sobreviva a discrepâncias de relógio entre máquinas. Esta necessidade materializa o §5.5 da norma.
3. **Utilidade no terreno.** O instrumento tem de funcionar num *smartphone*, com uma só mão, em redes móveis variáveis, sem instalação prévia de *software* especializado nem formação técnica avançada. Esta necessidade é uma condição de adopção: uma plataforma robusta mas inutilizável no momento da apreensão não resolve o problema.

A estas três necessidades técnicas acrescem requisitos legais e éticos não-negociáveis: cumprimento do RGPD, em particular do Artigo 32.ª (Parlamento Europeu e Conselho da União Europeia, 2016), e ausência de exposição de dados pessoais em ambiente *open-source*.

1.4 Objectivos

Objectivo geral. Desenvolver uma plataforma *web mobile-first* que digitalize a recolha, preservação e transferência de prova digital pelos *first responders* da PSP, com integridade verificável conforme a norma ISO/IEC 27037:2012, e que seja demonstrável num *smartphone* em condições reais.

Objectivos específicos. Os seis objectivos específicos da proposta correspondem às funcionalidades F1–F6 do MVP, com critérios de aceitação observáveis. Cada objectivo é mapeado, no Capítulo 3, ao seu estado actual de implementação e à evidência empírica que o suporta.

Obj.	Funcionalidade	Critério de aceitação observável
F1	Autenticação JWT com perfis distintos	Login válido em <2 s; perfis com permissões verificáveis; erro 401 sem fuga de informação interna.
F2	Registo de prova com fotografia e GPS	Criação completa em <30 s no *smartphone*; identificador único, *timestamp* do servidor, *hash* SHA-256 dos metadados e dos *bytes* da fotografia.
F3	Cadeia de custódia com máquina de estados imutável	Transições só na ordem canónica definida; sem retrocesso; *append-only* sem `UPDATE` nem `DELETE`; alteração directa na base de dados detectável por verificação de *hash*.

Obj.	Funcionalidade	Critério de aceitação observável
F4	Módulo de forense digital (ficha de dispositivo)	Ficha completa associada a uma evidência existente: tipo, marca, modelo, estado, número de série.
F5	API REST documentada	Mínimo dez *endpoints* testáveis via Swagger UI gerado automaticamente.
F6	Exportação de relatório de ocorrência em PDF	Documento contendo todos os registos e o histórico de custódia da ocorrência, juntável ao expediente físico.

Tabela 1: Objectivos específicos F1–F6 do MVP e respectivos critérios de aceitação observáveis.

1.5 Restrições

A execução do projecto está sujeita a quatro restrições documentadas em `docs/scope/risks.tex`: duração de 16 semanas com três marcos eliminatórios fixos (proposta, intercalar, final); trabalho estritamente individual sem possibilidade de divisão de tarefas; dependência de planos gratuitos da Neon (PostgreSQL gerido) e da Fly.io (alojamento Frankfurt) durante o desenvolvimento, com migração para *Neon Launch* (∼5 USD/mês) antes da defesa pública; e indisponibilidade de orçamento para serviços comerciais, *e.g. APIs* pagas de OCR ou OSINT.

A estas restrições acrescem condicionantes éticas: o sistema é desenhado para uso futuro com dados reais, mas é demonstrado exclusivamente com dados sintéticos gerados por *seed*; o repositório é público no GitHub e segue o princípio de "segurança por desenho, transparência por omissão".

1.6 Resultados esperados

A verificação do cumprimento dos objectivos é tripla: pelo funcionamento da aplicação em produção em forensiq.pt, pelos testes automatizados no repositório e pela validação externa de postura de segurança.

A aplicação em produção permite a um avaliador externo, nomeadamente o orientador, executar os fluxos críticos — criação de ocorrência, registo de evidência com GPS e fotografia, transição de custódia, exportação de PDF — sem necessidade de configurar ambiente local. Esta capacidade serve de demonstração assíncrona do MVP e é abordada em §3.5. A *suite* automatizada de testes unitários e de integração (Capítulo 3) exercita os caminhos críticos de imutabilidade, autorização e encadeamento de *hash*. Os três testes externos de segurança em `docs/compliance/external-tests/` (HSTS *Preload* submetido a Chromium, Mozilla, Edge e Apple; Mozilla HTTP Observatory A+; Qualys SSL Labs A+) validam, com auditoria independente, a postura criptográfica e os cabeçalhos HTTP.

1.7 Calendário planeado

A Tabela 2 apresenta o calendário das 16 semanas do projecto. Os marcos formais (Fase 1 a 25 mar, Fase 2 a 6 mai, Fase 3 a 24 jun) são fixos. As semanas intermédias foram executadas com pequenos ajustes face à proposta; em particular, a demo interna síncrona prevista para a Sem. 7 (28 abr–2 mai) não foi realizada nessa janela, sendo substituída por demonstração assíncrona em produção — assunto desenvolvido em §3.4 e comunicado ao orientador em paralelo a este relatório.

Sem.	**Datas**	**Conteúdo planeado**	**Marco**
1–2	17–28 mar	Proposta inicial, configuração do repositório.	Proposta (25 mar)
3–4	31 mar–11 abr	MoSCoW final, ADRs 0001–0005, modelo ER, primeiros *models* Django.	
5–6	14–25 abr	Autenticação JWT em *cookies* HttpOnly (ADR-0009), CSP *nonce-based* (ADR-0007), F2 e F3 funcionais.	
7	28 abr–2 mai	Integração de *triggers* PostgreSQL e ADRs 0006–0010 (a auditoria interna de segurança em `AUDIT_2026-04-16.md` e os *fixes B-C2/B-C3* de 19 abr antecederam esta semana, conforme §3.4). *Demo interna síncrona não realizada nesta janela.*	Demo interna
8	5–6 mai	Submissão do relatório intercalar.	**Intercalar (6 mai)**
9–10	7–16 mai	Reforço de cobertura de testes; revisão de UX em condições de terreno; eventual demo síncrona com o orientador.	
11–12	19–30 mai	Polimento da *interface*, módulo extra opcional (*e.g. timeline* agregada de auditoria).	
13	2–6 jun	Revisão geral do sistema; validação dos critérios de aceitação.	
14	9–13 jun	Redacção dos Capítulos 4 e 5 do relatório final; bibliografia APA; anexos.	
15	16–20 jun	Reunião de preparação para defesa; ensaio de perguntas de júri.	Prep. defesa
16	24 jun	Submissão do relatório final e do código.	**Final (24 jun)**

Sem.	Datas	Conteúdo planeado	Marco

Tabela 2: Calendário das 16 semanas com marcos formais sublinhados.

2 Desenho

Este capítulo apresenta a concepção do ForensiQ, organizada do nível mais agregado para o mais detalhado. A Secção 2.1 introduz a fronteira do sistema; §2.2 descreve a arquitectura segundo o modelo C4 (Brown, 2014) e sumariza as decisões registadas em ADRs; §2.3 é o núcleo técnico, dedicado ao algoritmo de certificação e integridade da prova; §2.4 aborda a segurança transversal; §2.5 apresenta a conformidade com a norma ISO/IEC 27037:2012 e o RGPD Art. 32; §2.6 fecha com a *interface* do utilizador. Cada afirmação técnica é ancorada num caminho `ficheiro:linha` verificável no repositório.

2.1 Visão geral da solução

ForensiQ é uma aplicação *web* servida pelo Django, acedida via *browser* a partir de *smartphone* ou computador. Existem três tipos de utilizadores: `AGENT` (agente PSP no terreno, cria ocorrências e regista evidências), `EXPERT` (perito forense, recebe a prova e participa nas transições de custódia) e o utilizador administrativo, marcado pela *flag* `is_staff` do Django (acesso ao *Django Admin* com prefixo configurável). Os perfis `AGENT` e `EXPERT` são distinguidos pelo campo `User.profile` (`src/backend/core/models.py:123-129`); um *staff* pode ter qualquer perfil operacional simultaneamente.

A solução tem dois sistemas externos relevantes: a *Geolocation API* do *browser*, usada com consentimento explícito para capturar GPS no momento da apreensão; e o serviço imeidb.xyz, usado opcionalmente para enriquecer fichas de telemóvel a partir do IMEI, com *cache* em PostgreSQL por 30 dias (ADR-0008). O *frontend* não é um contentor independente: por decisão registada em ADR-0004, é servido directamente pelo Django via *templates* HTML e estáticos, sem *build step*. A plataforma vive em produção em forensiq.pt, alojada em Fly.io Frankfurt com PostgreSQL gerido pela Neon na mesma região (ADR-0001 e ADR-0005).

2.2 Arquitectura

2.2.1 Diagrama C4 nível 1 — Contexto

A Figura 1 apresenta o sistema, os seus utilizadores e os sistemas externos com que interage, segundo o nível 1 do modelo C4 (Brown, 2014). O fonte do diagrama está em `docs/architecture/diagrams/c4-context.mmd`.

2.2.2 Diagrama C4 nível 2 — Contentores

A Figura 2 decompõe a aplicação em três contentores principais: a aplicação Django (servindo simultaneamente *templates* HTML e API REST), a base de dados PostgreSQL gerida pela

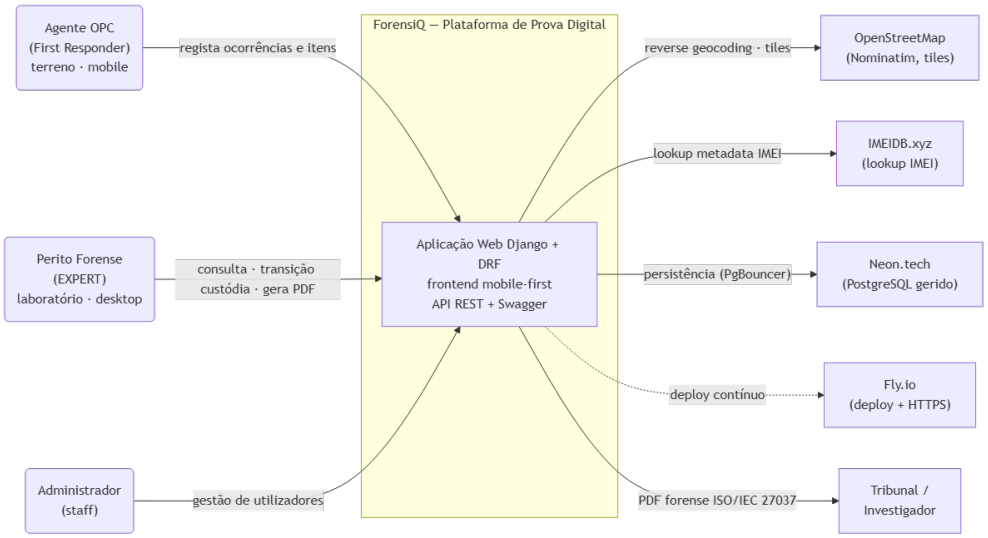

Figura 1: Diagrama C4 nível 1: ForensiQ, utilizadores e sistemas externos.

Neon, e o volume de *media* persistente da Fly.io para fotografias. O fonte do diagrama está em `docs/architecture/diagrams/c4-container.mmd`.

2.2.3 Stack tecnológica

A Tabela 3 resume a *stack* adoptada e a justificação sucinta de cada escolha. As decisões de fundo estão registadas em ADRs próprios e são sumarizadas em §2.2.5.

Camada	**Tecnologia**	**Motivo da escolha (ADR)**
Backend web	Django 5 + DRF 3.15	ORM maduro, *admin* integrado, autenticação *built-in*; *ecosystem* convencional para defesa académica (ADR-0002, ADR-0003).
Documentação OpenAPI	drf-spectacular 0.27	Swagger UI gerado automaticamente a partir dos *viewsets*; cumpre o RF05.
Autenticação	SimpleJWT 5 + cookies HttpOnly	JWT *stateless* interoperável; transporte em *cookies* mitiga XSS (ADR-0009).
Validação de imagens	Pillow 12	Suporte EXIF e validação MIME no *upload*.
Geração de PDF	ReportLab 4	Geração programática conforme *layout* forense, sem dependências externas.
Cliente HTTP *server-side*	httpx 0.27	Cliente moderno usado no *lookup* IMEI (ADR-0008).
Filtragem	django-filter 24	Filtros declarativos por *viewset*, integrados na API.

Camada	Tecnologia	Motivo da escolha (ADR)
Base de dados	PostgreSQL via Neon.tech	SLA gerido, *branching*, *point-in-time recovery* 7 dias, plano gratuito adequado ao MVP (ADR-0001).
Frontend	HTML5 + CSS3 + JavaScript *vanilla*	Sem *build step*; *mobile-first*; explicabilidade total a um júri (ADR-0004).
Mapa	Leaflet 1.9 + OpenStreetMap	Sem chave de API; *tiles* OSM com Referrer-Policy `strict-origin` (ADR-0007).
Alojamento	Fly.io fra	Frankfurt (próximo da Neon); HTTPS Let's Encrypt automático; volume persistente (ADR-0005).
Servidor WSGI	Gunicorn 22 + WhiteNoise 6	Dois *workers*; estáticos servidos directamente, sem CDN.
Diagramas	Mermaid + C4-PlantUML	*Diagrams-as-code* versionados em `docs/architecture/diagrams/`.

Tabela 3: Pilha tecnológica do ForensiQ com justificação por camada.

2.2.4 Modelo de dados

A Figura 3 apresenta o modelo entidade-relação. As entidades centrais são `User`, `Occurrence`, `Evidence`, `DigitalDevice`, `ChainOfCustody` e `AuditLog`, todas implementadas em `src/backend/core/models.py`.

User (`models.py:116-158`). Modelo Django `AbstractUser` estendido com `profile` (escolhas `AGENT`, `EXPERT`), `badge_number` e `phone`. As permissões da API consultam o `profile` via *permission classes* dedicadas em `permissions.py`.

Occurrence (`models.py:164-273`). Caso ou cena de crime, com `number` (NUIPC; o seed sintético usa um formato informativo do tipo `NUIPC.812/2026.LISBOA` para legibilidade em demo, distinto do formato canónico português `NNN/AA.X[A-Z]+` que circularia em produção sobre dados reais), `code` legível auto-gerado (`OCC-YYYY-NNNNN`), `address`, coordenadas GPS, `date_time` e *foreign key* para o agente criador.

Evidence (`models.py:306-724`). Item apreendido, com taxonomia de 18 tipos digital-first (ADR-0010): 14 raízes (`MOBILE_DEVICE`, `COMPUTER`, `STORAGE_MEDIA`, `GAMING_CONSOLE`, `GPS_TRACKER`, `SMART_TAG`, `CCTV_DEVICE`, `VEHICLE`, `DRONE`, `IOT_DEVICE`, `NETWORK_DEVICE`, `DIGITAL_FILE`, `RFID_NFC_CARD`, `OTHER_DIGITAL`) e 4 sub-componentes típicos (`SIM_CARD`,

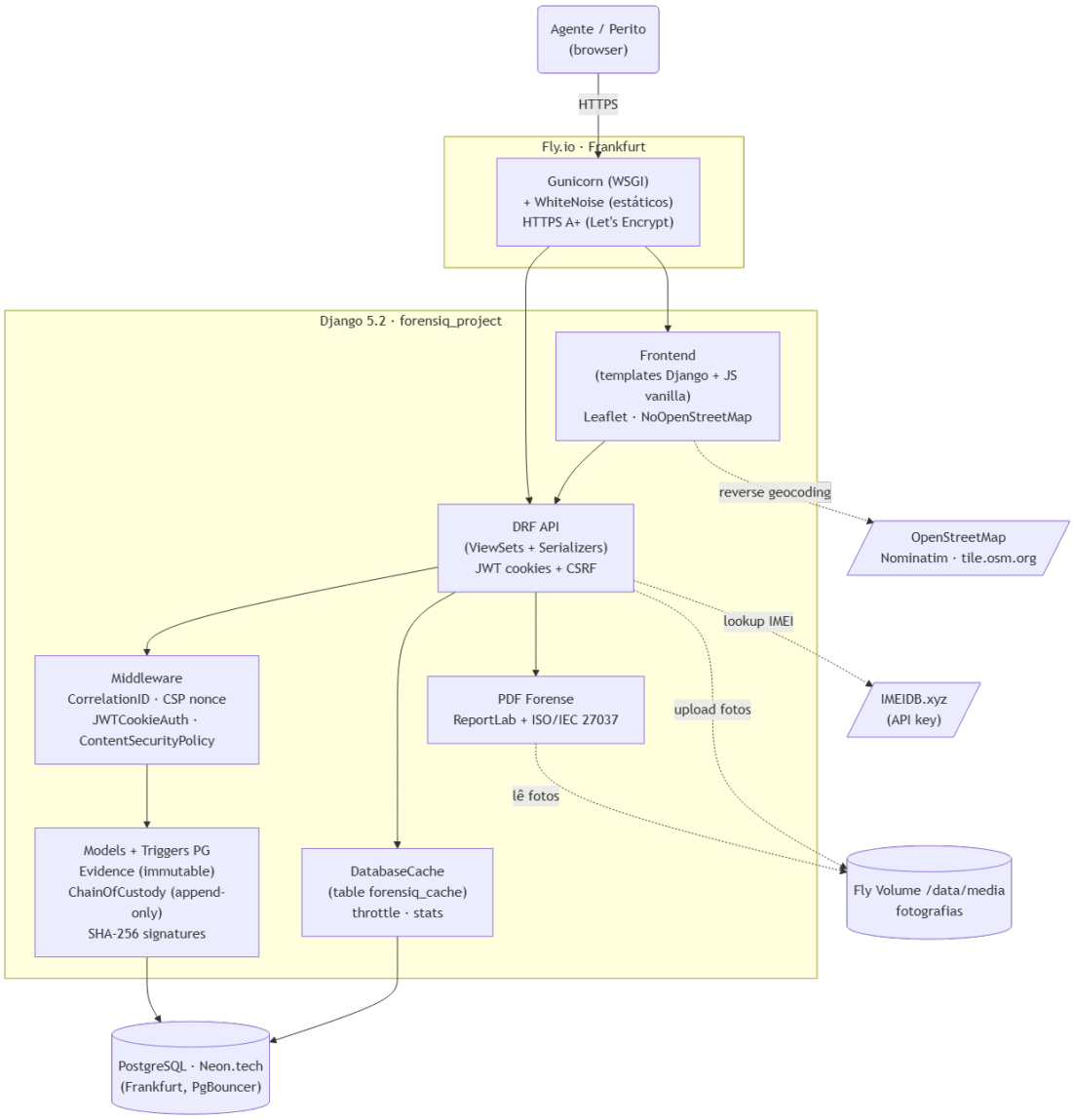

Figura 2: Diagrama C4 nível 2 (contentores) com tecnologias principais.

`MEMORY_CARD`, `INTERNAL_DRIVE`, `VEHICLE_COMPONENT`). A relação `parent_evidence` permite hierarquia até três níveis (`MAX_TREE_DEPTH=3`, `models.py:318`); validação em `Evidence.clean` impede ciclos e migrações entre ocorrências. Cada evidência tem `integrity_hash` SHA-256 calculado no `save`, `type_specific_data` em `JSONField` (com validadores dedicados de IMEI, IMSI, ICCID, MAC, VIN), fotografia opcional, GPS e número de série.

DigitalDevice (`models.py:730-851`). Ficha técnica detalhada *one-to-one* com uma `Evidence`. Coexiste com a hierarquia de sub-componentes da `Evidence` por razões de compatibilidade (ADR-0010).

ChainOfCustody (`models.py:858-1114`). Núcleo técnico do projecto, abordado em detalhe em §2.3.

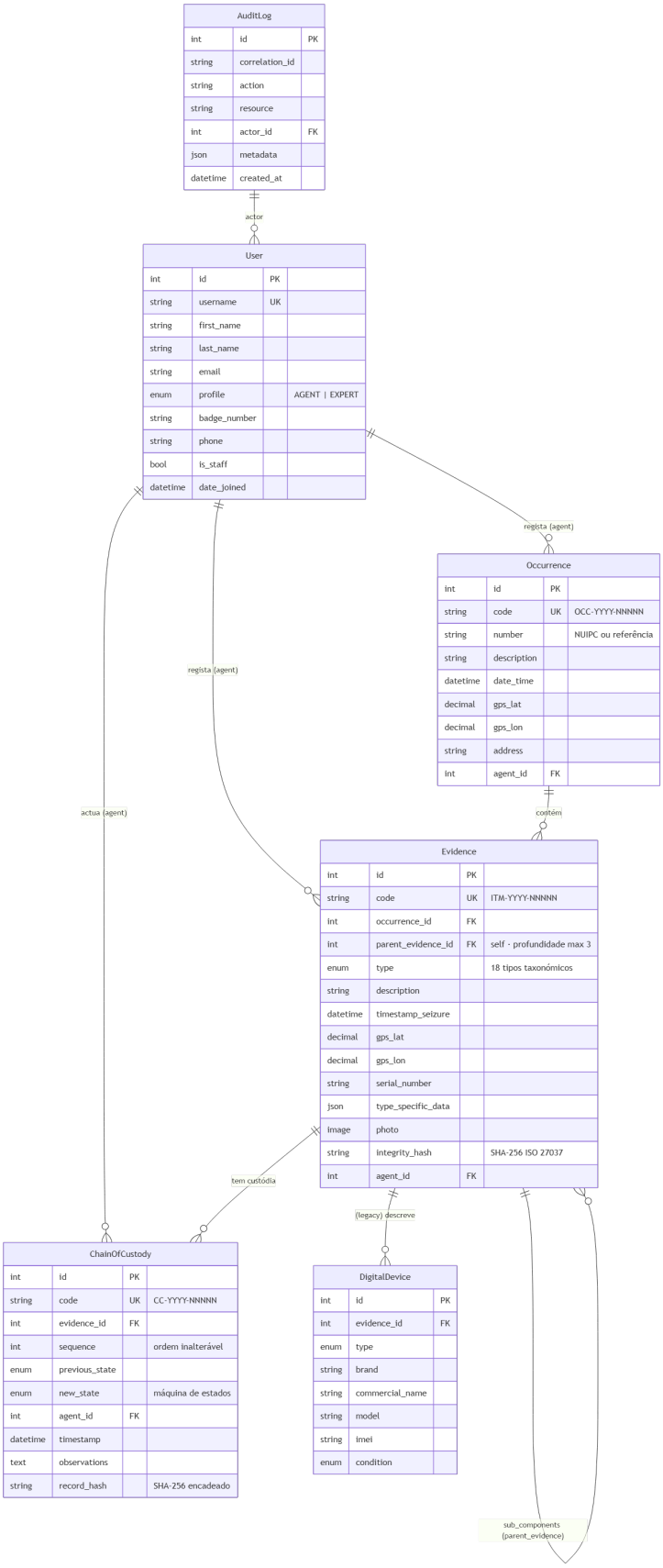

Figura 3: Modelo entidade-relação do ForensiQ.

AuditLog (`models.py:1121-1259`). Registo transversal de acessos e operações sensíveis: `user`, `action`, `resource_type`, `resource_id`, `ip_address`, `correlation_id` (UUID por pedido HTTP) e `timestamp` do servidor.

2.2.5 Decisões de arquitectura

As decisões estruturantes foram registadas como ADRs versionados em `docs/architecture/adr/`. A Tabela 4 sumariza as dez decisões em vigor; o detalhe completo, com alternativas rejeitadas, está nos ficheiros ADR.

ADR	Tema	Decisão e principal alternativa rejeitada
0001	PostgreSQL + Neon	Postgres com *triggers* `BEFORE UPDATE/DELETE` para imutabilidade ao nível da BD; alojado na Neon free, plano *Launch* antes da defesa. Rejeitadas: Supabase (pausa após 7 dias), Render (expira 30 dias), SQLite (sem *triggers*).
0002	Estrutura do projecto Django	Django 5 + DRF; `User AbstractUser` com `profile`; `ChainOfCustody` *append-only*. Rejeitada: FastAPI (sem ORM nem *admin* integrados).
0003	Desenho da API REST	DRF *ModelViewSet* + *DefaultRouter*; permissões granulares por perfil; sem PUT/PATCH/DELETE em `ChainOfCustody`.
0004	Arquitectura do *frontend*	HTML/CSS/JS *vanilla* servido pelo Django. Rejeitada: SPA React (sobrecusto de *build* e *bundle*).
0005	*Deployment* em produção	Fly.io fra + Neon Postgres + IPv4 dedicado + Let's Encrypt automático. Rejeitadas: Render (sem fra; *cold start*), Railway (apenas Amesterdão).
0006	Extensibilidade modular	*Apps* Django independentes para futuros módulos forenses; *services layer* para *hash*, exportação e auditoria. Rejeitada: herança de modelo (*multi-table inheritance*).
0007	SRI e Referrer-Policy	Manter SRI nos recursos do CDN com *hashes* SHA-512 actualizados; `strict-origin-when-cross-origin`. Rejeitada: remover SRI (perde defesa em profundidade).
0008	*Cache* para *lookups* IMEI/VIN	`DatabaseCache` em Neon Postgres, TTL 30 dias para IMEI, 180 para VIN. Rejeitadas: `LocMemCache` (perde-se em *restart*), Redis externo (custo).

ADR	Tema	Decisão e principal alternativa rejeitada
0009	JWT em *cookies* HttpOnly	Transporte em `fq_access`/`fq_refresh` HttpOnly + Secure + SameSite=Strict; CSRF *double-submit*. Rejeitada: `Authorization: Bearer` em `localStorage`.
0010	Taxonomia de evidência	18 códigos *digital-first* alinhados com ISO/IEC 27037 e ENFSI 2024; hierarquia até três níveis; `type_specific_data` em `JSONField`. Rejeitada: importação 1:1 da taxonomia Cellebrite UFED.

Tabela 4: Síntese dos dez ADRs em vigor; detalhe em `docs/architecture/adr/`.

2.3 Algoritmo de certificação e integridade

A integridade da prova digital é o requisito mais crítico do ForensiQ. Esta secção apresenta, do mais geral ao mais específico, o problema adversarial, a justificação do algoritmo SHA-256, as duas fórmulas determinísticas implementadas, o tratamento da concorrência e a defesa em três camadas que torna a adulteração detectável mesmo perante um atacante com acesso directo à base de dados.

2.3.1 Modelo de ameaças aplicado

A análise seguiu o método STRIDE simplificado para um sistema de gestão de prova digital sem aquisição forense, sem armazenamento distribuído e com um único *tenant*. A definição operacional adoptada de *prova digital adulterada* é: qualquer registo armazenado cujo conjunto de campos canónicos não reproduza o *hash* SHA-256 declarado, ou cuja sequência de custódia não reproduza o encadeamento de *hashes* a partir da raiz da cadeia. A detecção é por recálculo independente, conforme ISO/IEC 27037 §5.4–5.5 (ISO/IEC, 2012).

Delimitação honesta do modelo. A função de *hash* usada é SHA-256 sobre campos públicos, sem chave secreta (não-HMAC). O modelo presume, portanto, que o controlo de acesso ao motor PostgreSQL — nomeadamente a contas com privilégio de `ALTER TABLE` — é organizacional e fica fora do âmbito da plataforma. Um adversário com privilégios DBA conseguiria desactivar *triggers*, alterar registos e recomputar a cadeia internamente; a verificação ficaria internamente consistente mas divergiria de qualquer cópia auditada anteriormente. Esta limitação é coerente com o modelo de verificação por *recálculo independente* previsto pela ISO/IEC 27037: requer que exista uma âncora externa anterior (cópia em backup auditado, exportação assinada, ou selo *Time-Stamping Authority* RFC 3161) para detectar a adulteração com privilégios de motor. A âncora externa é trabalho futuro e está identificada como tal em §3.4; o MVP entrega a cadeia internamente verificável e a sua publicação em exportações imutáveis (PDF e CSV assinados) é o caminho de endurecimento previsto para a Fase 3.

2.3.2 Escolha do algoritmo de *hash*

A integridade dos registos forenses do ForensiQ assenta numa função de *hash* criptográfica. A norma ISO/IEC 27037 não impõe um algoritmo específico, mas exige resistência à colisão, resistência à pré-imagem e maturidade reconhecida. As alternativas comparadas foram MD5 (quebrado, com colisões em segundos), SHA-1 (quebrado em 2017 com a colisão *SHAttered*), SHA-3/Keccak (vigente, mas com menos jurisprudência forense), BLAKE2/3 (rápidos, pouca adopção forense) e SHA-256 (vigente, FIPS 180-4, sem ataques práticos conhecidos, jurisprudência forense estabelecida). A decisão pelo SHA-256 fundamenta-se em quatro critérios: maturidade e normalização (FIPS 180-4 com mais de uma década), jurisprudência forense (referência da ENFSI e da ACPO (Association of Chief Police Officers, 2012)), interoperabilidade (verificável com `sha256sum` ou `certutil` em qualquer sistema operativo, sem ferramentas especializadas), e custo computacional desprezável para o volume previsto. A implementação usa `hashlib.sha256` da biblioteca *standard* do Python, sem dependências de terceiros.

2.3.3 Hash de integridade do registo de evidência

Cada registo de evidência recebe, no momento da criação, um *hash* SHA-256 que cobre todos os campos canónicos mais o conteúdo binário da fotografia anexa. A função `compute_integrity_hash` (`models.py:554-584`) é determinística e reprodutível. A Listagem 1 apresenta a versão sintética.

```python
def compute_integrity_hash(self):
    photo_hash = self._compute_photo_hash()  # SHA-256 dos bytes da imagem
    tsd_json = json.dumps(
        self.type_specific_data or {},
        sort_keys=True, separators=(',', ':'), ensure_ascii=False,
    )
    data = (
        f'{self.occurrence_id}|{self.type}|{self.parent_evidence_id or ""}|'
        f'{self.description}|{self.gps_lat}|{self.gps_lon}|'
        f'{self.timestamp_seizure.isoformat()}|{self.serial_number}|'
        f'{self.agent_id}|tsd={tsd_json}|photo={photo_hash}'
    )
    return hashlib.sha256(data.encode('utf-8')).hexdigest()
```

Listagem 1: `compute_integrity_hash` (`models.py:554-584`).

Três aspectos são essenciais: o `type_specific_data` é serializado com `sort_keys=True`, garantindo representação canónica independente da ordem de inserção das chaves; a fotografia entra na fórmula via o seu próprio *hash* SHA-256 (`_compute_photo_hash`, `models.py:534-552`), de modo a que a substituição dos *bytes* da imagem com manutenção do nome do ficheiro — vector clássico de adulteração — seja imediatamente detectada; o *timestamp* usado é o de apreensão (`timestamp_seizure`), o momento forense relevante.

2.3.4 Cadeia de custódia com *hash* encadeado

Cada transição de estado é registada como entrada *append-only* cujo *hash* SHA-256 inclui o *hash* do registo anterior — estrutura inspirada em *blockchain*, sem prova-de-trabalho. A Figura 4 apresenta o fluxo de cálculo e a propagação da invalidação por adulteração.

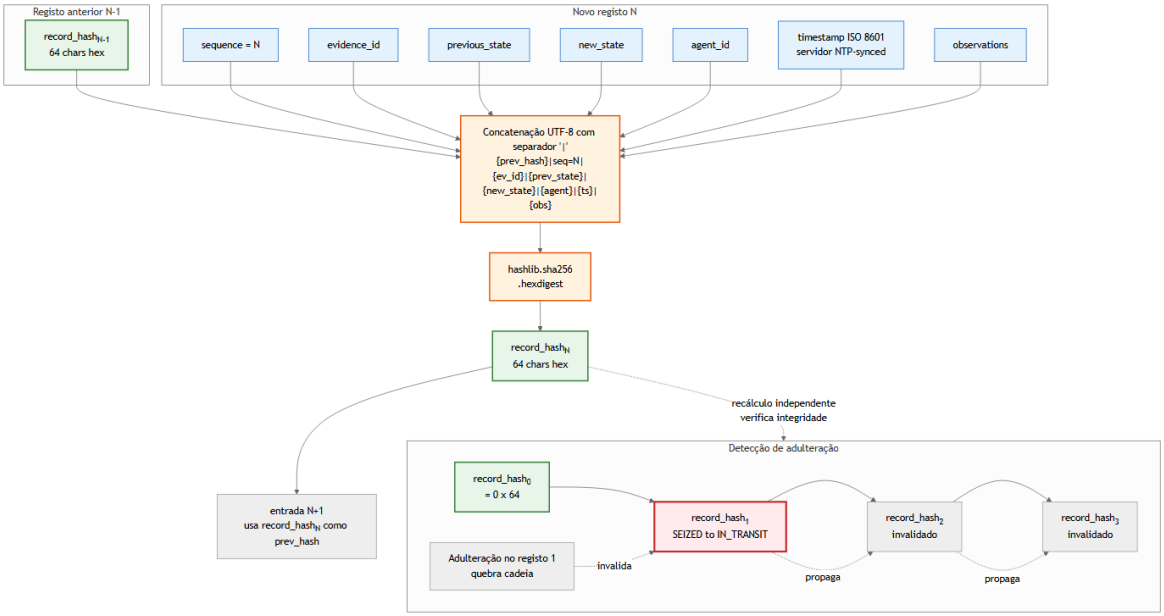

Figura 4: Fluxograma do cálculo do *hash* encadeado e da detecção de adulteração.

A função `compute_record_hash` (`models.py:989-1031`) é *pura* e *determinística*: recebe o *hash* anterior como parâmetro e calcula o resultado a partir exclusivamente dos campos públicos do registo. Esta pureza permite verificabilidade externa — qualquer perito independente, com acesso aos campos e ao *hash* da raiz da cadeia, pode reproduzir todos os *hashes* subsequentes.

Fórmula canónica.

$$\begin{aligned}\text{record_hash}_n = \text{SHA-256}(&\text{prev_hash}_{n-1} \parallel \text{seq} \parallel \text{evidence_id} \\ &\parallel \text{prev_state} \parallel \text{new_state} \parallel \text{agent_id} \\ &\parallel \text{ts}_{\text{ISO}} \parallel \text{obs})\end{aligned}$$

onde $\parallel$ denota concatenação UTF-8 separada por "`|`", $\text{prev_hash}_0 = 0 \times 64$ é o *hash* convencional da raiz, e $\text{seq} = 1, 2, 3, \ldots$ é o número sequencial monótono dentro da evidência, garantido por `UniqueConstraint(evidence, sequence)` (`models.py:957-962`).

Sequência canónica vs. *timestamp*. A escolha de `sequence` (não do *timestamp*) como identificador ordinal é deliberada. *Timestamps* podem sofrer derivações entre máquinas, em particular sob dessincronização NTP, e dois registos no mesmo segundo poderiam ambiguar a ordem. A `UniqueConstraint(evidence, sequence)` é o invariante forense: para cada evidência, existe no máximo um registo com cada número sequencial.

2.3.5 Concorrência e atomicidade

A escrita de uma transição exige a leitura prévia do último registo da evidência seguida da inserção do novo. Sem cuidado, esta sequência expõe duas *races*: dois agentes em escrita concorrente podem produzir entradas com a mesma `sequence` ou *hashes* encadeados ao mesmo predecessor.

A solução combina três mecanismos PostgreSQL. **Primeiro**, `transaction.atomic()` envolve toda a operação numa transacção; falha em qualquer ponto reverte tudo. **Segundo**, `select_for_update()` adquire um *row lock* pessimista sobre o último registo, forçando escritores concorrentes a aguardar. **Terceiro**, a `UniqueConstraint(evidence, sequence)` rejeita uma sequência duplicada com `IntegrityError` mesmo perante *bug* aplicacional. A Listagem 2 apresenta o esqueleto.

```python
def save(self, *args, **kwargs):
    if self.pk is not None:
        raise ValidationError('Registos imutáveis')
    with transaction.atomic():
        # Lock pessimista na Evidence parent: cobre o caso degenerado
        # em que a cadeia ainda não tem registos e .first() devolveria
        # None sem adquirir lock (correcção pós-auditoria, B-C4).
        Evidence.objects.select_for_update().filter(pk=self.evidence_id).
            first()
        last_record = (
            ChainOfCustody.objects
            .select_for_update()
            .filter(evidence=self.evidence)
            .order_by('-sequence')
            .first()
        )
        self.previous_state = last_record.new_state if last_record else ''
        self.sequence = (last_record.sequence + 1) if last_record else 1
        self.timestamp = timezone.now()
        self.full_clean()
        previous_hash = last_record.record_hash if last_record else '0' * 64
        self.record_hash = self.compute_record_hash(previous_hash=
            previous_hash)
        super().save(*args, **kwargs)
```

Listagem 2: Inserção atómica em `ChainOfCustody.save` (`models.py:1044-1107`).

A passagem explícita de `previous_hash` a `compute_record_hash` (correcção da auditoria interna de 19 abr 2026, identificada como *B-C2*) elimina uma segunda *query* fora do *lock*, mantendo a função *pura*. O `full_clean()` antes do `super().save()` delega na `ChainOfCustody.clean` (`models.py:973-987`) a validação da transição contra `VALID_TRANSITIONS` (`models.py:883-892`), recusando saltos como `APREENDIDA` → `EM_PERICIA`. O lock pessimista adicional na `Evidence` parent fecha o caso degenerado em que a cadeia ainda não tem registos: sem ele, dois pedidos concorrentes a criar a primeira transição obteriam ambos `None` de `.first()` (sem *row* para bloquear), gerando duas inserções com `sequence=1` cuja segunda só seria rejeitada tarde, pela `UniqueConstraint`,

com `IntegrityError` não tratado.

2.3.6 Imutabilidade em três camadas

A integridade de uma transição, depois de gravada, é preservada por defesa em profundidade de três camadas independentes: ORM Django, DRF e *triggers* PostgreSQL. Cada camada presume que as anteriores podem ter falhado ou sido contornadas. A Figura 5 ilustra os três caminhos de ataque considerados.

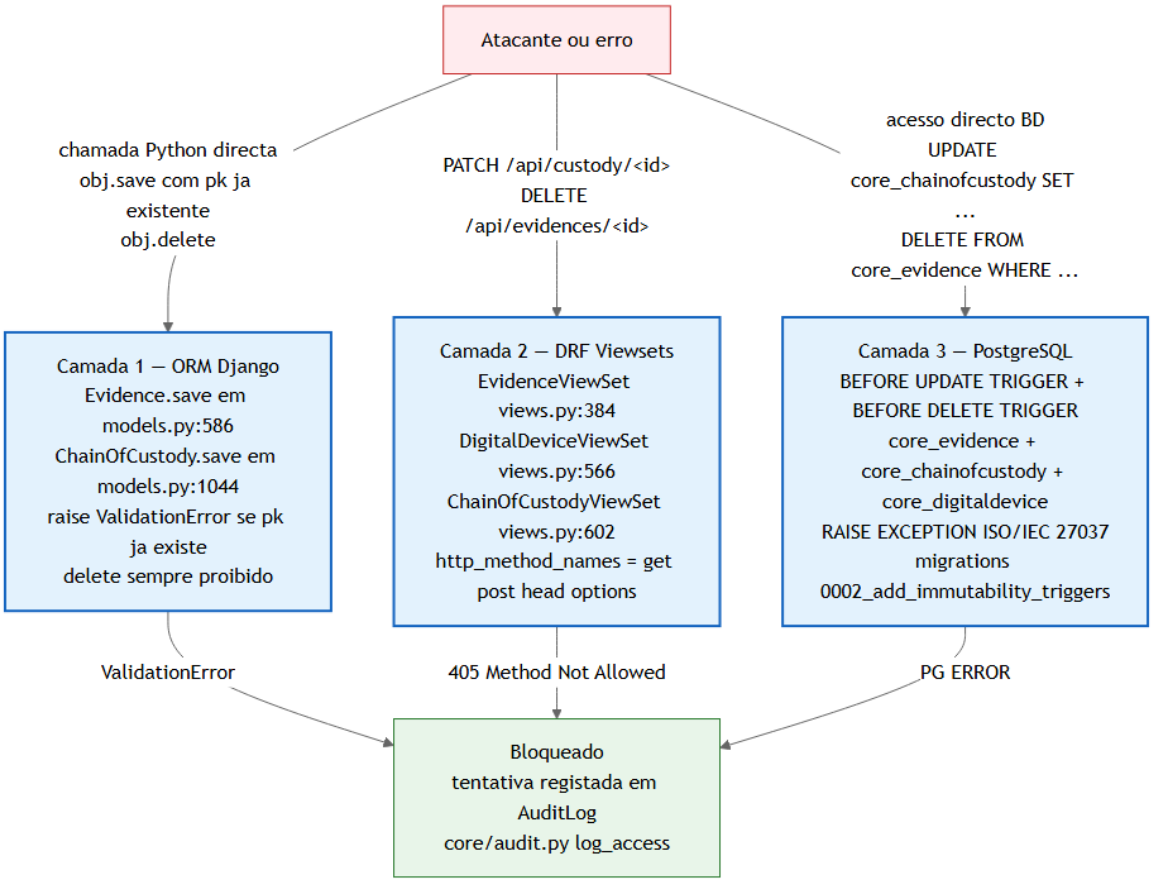

Figura 5: Defesa em profundidade contra adulteração: ORM, DRF e *triggers* PostgreSQL.

Camada 1 — ORM Django. Os métodos `Evidence.save` (`models.py:586`), `Evidence.delete` (`models.py:619`), `ChainOfCustody.save` (`models.py:1044`) e `ChainOfCustody.delete` (`models.py:1109`) interceptam chamadas Python e lançam `ValidationError` sempre que se detecta tentativa de actualização (`pk is not None`) ou eliminação. Esta camada protege contra erros do próprio código aplicacional.

Camada 2 — Django REST Framework. `EvidenceViewSet` (`views.py:365`, `http_method_names` em `views.py:384`), `DigitalDeviceViewSet` (`views.py:554`, `http_method_names` em `views.py:566`) e `ChainOfCustodyViewSet` (`views.py:588`, `http_method_names` em `views.py:602`) declaram `http_method_names = ['get', 'post', 'head', 'options']`; o efeito visível para um cliente que tente `PUT`, `PATCH` ou `DELETE` é uma resposta `405 Method Not Allowed`.

Camada 3 — *Triggers* PostgreSQL. A última linha de defesa é uma *trigger* PL/pgSQL aplicada directamente à tabela, que rejeita `UPDATE` e `DELETE` mesmo perante *queries* SQL emitidas fora da aplicação. A Listagem 3 é o SQL exacto, conforme `src/backend/core/migrations/0002_add_immutability_triggers.py:63-82`.

```
CREATE OR REPLACE FUNCTION prevent_custody_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'Registos de cadeia de custódia são imutáveis (ISO/IEC 27037). '
        'Operação bloqueada: %', TG_OP;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_custody_no_update
    BEFORE UPDATE ON core_chainofcustody
    FOR EACH ROW EXECUTE FUNCTION prevent_custody_modification();
CREATE TRIGGER trg_custody_no_delete
    BEFORE DELETE ON core_chainofcustody
    FOR EACH ROW EXECUTE FUNCTION prevent_custody_modification();
```

Listagem 3: *Triggers* PostgreSQL em `core_chainofcustody` (*migration* `0002`).

A mesma *migration* instala *triggers* análogas para `core_evidence` e `core_digitaldevice`. A *migration* `0013_protect_occurrence.py` estende a defesa a `core_occurrence` com a mesma estrutura (*trigger* `trg_occurrence_no_update` e `trg_occurrence_no_delete`), fechando a janela em que os campos de uma ocorrência (NUIPC, morada, GPS, *timestamp*) eram mutáveis sem invalidar o `integrity_hash` das evidências referenciadas. A integridade está assim materializada em quatro tabelas: `core_evidence`, `core_chainofcustody`, `core_digitaldevice` e `core_occurrence`.

2.4 Segurança transversal

2.4.1 Autenticação JWT em *cookies* HttpOnly

A autenticação assenta em *tokens* JWT (Sage, 2024) transportados em *cookies* `HttpOnly + Secure + SameSite=Strict`, conforme decisão registada em ADR-0009 e implementação em `src/backend/core/auth.py` (89 linhas). A escolha do transporte por *cookie* (em vez de `Authorization: Bearer` em `localStorage`) mitiga especificamente a *exfiltração* do *token* via XSS: um *cookie* `HttpOnly` é opaco ao JavaScript da página, pelo que a injecção bem-sucedida de um *script* numa vulnerabilidade de XSS não permite copiar o *token* para um servidor remoto. A mitigação da própria injecção XSS, que poderia ainda assim invocar *API* de mesma origem com o *cookie* anexado pelo *browser*, é tratada em camada anterior pelo CSP nível 3 com *nonce* por pedido e ausência de `unsafe-inline` ou `unsafe-eval` (§2.4.3).

Quatro detalhes são essenciais: *(i)* os *cookies* `fq_access` (validade configurável, default 60 minutos) e `fq_refresh` (default 7 dias) são colocados pelo `set_auth_cookies` (`auth.py:71-84`) com `httponly=True`, `secure=not settings.DEBUG` e `samesite='Strict'`; o *cookie* `csrftoken`, único não-`HttpOnly`, é o *token* CSRF que o *frontend* replica

em `X-CSRFToken` para pedidos não-seguros. *(ii)* A classe `JWTCookieAuthentication` (`auth.py:33-58`) lê o *token* do cabeçalho ou, na sua ausência, do *cookie* `fq_access`; valida-o criptograficamente; e em pedidos com método não-seguro invoca `enforce_csrf` para validar `X-CSRFToken`. *(iii)* O *refresh* é rotativo com *blacklist* (`ROTATE_REFRESH_TOKENS=True`, `BLACKLIST_AFTER_ROTATION=True` em `settings.py:180-181`), reduzindo a janela de exploração em caso de roubo. *(iv)* Os três *endpoints* de autenticação têm `AuthRateThrottle` de cinco pedidos por minuto, configurado em `REST_FRAMEWORK['DEFAULT_THROTTLE_RATES']`, para travar força bruta.

2.4.2 Autorização baseada em papéis (RBAC)

O modelo de autorização é granular por papel e por recurso, conforme ADR-0003. Existem dois papéis operacionais (`AGENT` e `EXPERT`, em `User.profile`) mais a *flag* `is_staff` para administração. Cinco classes de permissão implementadas em `src/backend/core/permissions.py`: `IsAgent`, `IsExpert`, `IsAgentOrExpert`, `IsOwnerOrReadOnly` e a classe-padrão `IsAuthenticated` do DRF.

A protecção contra IDOR (*Insecure Direct Object Reference*) é adicional: o `get_queryset()` de cada *viewset* restringe os recursos ao utilizador autenticado quando o perfil é `AGENT`; a classe `IsOwnerOrReadOnly` valida *ownership* em escritas. O conjunto `AuthorizationIDORTest` em `tests_api.py` exercita o caminho cruzado agente A *vs.* agente B.

2.4.3 Cabeçalhos HTTP de segurança

A defesa em camada HTTP combina *settings* Django (em produção, `DEBUG=False`) e o *middleware* dedicado `ContentSecurityPolicyMiddleware` (`middleware.py:68-155`). Os controlos principais são:

- `Strict-Transport-Security` (HSTS) com `max-age=31536000` (um ano), `includeSubDomains` e `preload` (`settings.py:284-286`). O domínio `forensiq.pt` foi submetido à lista *preload* dos principais *browsers* (validação externa em §2.5.3).
- `Content-Security-Policy` de nível 3, gerada por pedido com 11 directivas activas, materializadas em `middleware.py:122-140`. As principais são: `default-src 'self'`; `script-src 'self' 'nonce-{nonce}' https://cdnjs.cloudflare.com` (sem `unsafe-inline` nem `unsafe-eval`); `style-src 'self' 'nonce-{nonce}' https://cdnjs.cloudflare.com https://fonts.googleapis.com`; `connect-src 'self' https://nominatim.openstreetmap.org`; `img-src 'self' data: https://*.tile.openstreetmap.org https://cdnjs.cloudflare.com`; `font-src 'self' https://cdnjs.cloudflare.com https://fonts.gstatic.com`; `object-src 'none'`; `frame-src 'none'`; `base-uri 'self'`; `frame-ancestors 'none'`; `form-action 'self'`; `upgrade-insecure-requests`. O *nonce* é único por pedido (`secrets.token_urlsafe(16)`, ∼22 chars / 128 bits de entropia). Em desenvolvimento (`DEBUG=True`) usa-se `Report-Only` para preservar o ciclo de desenvolvimento.

Fragilidade conhecida — conflito CSP3 com Leaflet. Em CSP nível 3, a directiva `style-src` controla também os atributos `style="..."` injectados dinamicamente no DOM via `element.style.x = ...`, salvo se for declarada a directiva derivada `style-src-attr`. O Leaflet 1.9.4, usado para o mapa de georreferenciação de ocorrências, posiciona os *tiles* via `style="transform: translate3d(...)"` aplicado por JavaScript em tempo de execução — comportamento incompatível com o `style-src` estrito (*nonce-only*) actualmente em vigor em produção. A correcção planeada para a iteração de 7–16 mai (antes do relatório final) é cirúrgica: adicionar `style-src-attr 'unsafe-inline'` à política, relaxando exclusivamente o vector dos atributos de estilo e preservando o resto da postura — `<style>` *blocks* continuam a exigir *nonce*, e `script-src` mantém-se sem `unsafe-inline` nem `unsafe-eval`. O risco residual aberto (injecção de `<div style="background:url(externa)">` por XSS) fica contido por `connect-src 'self'` e pela ausência de execução de JavaScript na regra. A separação `style-src-attr`/`style-src-elem` introduzida pelo CSP 3 foi precisamente desenhada para este caso de uso. Reconhece-se a fragilidade aqui em vez de a ocultar, conforme orientação do *Guia* (Pestana, 2026).

- `Referrer-Policy: strict-origin-when-cross-origin` (ADR-0007) — envia apenas a origem em pedidos *cross-origin*, satisfazendo a política do OpenStreetMap sem expor *paths* internos.
- `X-Content-Type-Options: nosniff`, `Cross-Origin-Opener-Policy: same-origin`, `Cross-Origin-Resource-Policy: same-origin` e `Permissions-Policy` restritiva.
- *Subresource Integrity* (SRI) SHA-512 nos recursos do CDN da Cloudflare (Leaflet 1.9.4), declarado em `_leaflet_head.html` e `_leaflet_js.html` (ADR-0007).

2.4.4 Outras defesas

Limitação de débito via `DEFAULT_THROTTLE_RATES`: `anon` 10/min, `user` 60/min, `auth` 5/min, `evidence_upload` 20/min, `pdf_export` 30/min, `csv_export` 10/min, `schema` 30/min (`settings.py:142-150`). Os três *endpoints* de autenticação em `auth_views.py` (`CookieLoginView`, `CookieRefreshView`, `CookieLogoutView`) usam o *scope* `auth` (5/min). **Paginação obrigatória** via `BoundedPageNumberPagination` (default 50, máximo 100, `pagination.py`). **Ocultação do *Django Admin*** por prefixo configurável via variável de ambiente `ADMIN_URL_PREFIX`, lida em `forensiq_project/urls.py:62`; é segurança por obscuridade que complementa, não substitui, a autenticação.

2.5 Conformidade ISO/IEC 27037 e RGPD Art. 32

2.5.1 Mapeamento à norma ISO/IEC 27037:2012

A norma ISO/IEC 27037:2012 (ISO/IEC, 2012) estabelece princípios e controlos para a identificação, recolha, aquisição e preservação de prova digital. A Tabela 5 resume o mapeamento entre as secções relevantes da norma e os mecanismos do ForensiQ; o documento completo de rastreabilidade está em `docs/scope/iso27037-traceability.tex` v1.2 (3 mai 2026).

Secção	Princípio da norma	Implementação no ForensiQ
§5.1	Princípios gerais; integridade, auditabilidade	RBAC com perfis `AGENT`/`EXPERT` (§2.4.2); JWT rotativo (§2.4.1); `AuditLog` com `correlation_id`.
§5.2	Identificação de prova digital	`Evidence` com 18 tipos digital-first (ADR-0010), descrição, `serial_number`, GPS, *timestamp* de apreensão, *hash*.
§5.3	Recolha e aquisição	`integrity_hash` cobre metadados *e bytes* da fotografia (§2.3.3); GPS no *frontend* via *Geolocation API*.
§5.4	Preservação de integridade (3 camadas)	*Append-only* no ORM (§2.3.6); *viewsets* com `http_method_names` restrito; *triggers* PostgreSQL na *migration* 0002.
§5.5	Cadeia de custódia	Máquina de estados linear de 7 estados PT (§2.6.4); *record_hash* encadeado (§2.3.4).
§6.1–6.2	Competência e documentação	RBAC fino; PDF forense gerado por `pdf_export.py` com declaração ISO/IEC 27037 incorporada.
§7.1–7.2	Dispositivos e suportes	`DigitalDevice` (ficha técnica) e taxonomia detalhada de suportes em `Evidence.type`; campos específicos via `type_specific_data` (JSON determinístico).

Tabela 5: Mapeamento sintético da ISO/IEC 27037:2012 aos mecanismos do ForensiQ.

2.5.2 Conformidade com o RGPD Art. 32

O Artigo 32.ª do RGPD (Parlamento Europeu e Conselho da União Europeia, 2016) exige medidas técnicas e organizacionais adequadas ao risco. A Tabela 6 apresenta o sumário executivo, com identificação honesta das lacunas relevantes; o tratamento detalhado está em `docs/compliance/rgpd-art32-checklist.md`.

Alínea	Domínio	Estado	Lacunas a endereçar
a)	Pseudonimização e cifragem	Parcial	`media/` sem cifra ao nível da aplicação.
b)	CIA-R	Parcial	Validação a generalizar a todos os modelos.

Alínea	Domínio	Estado	Lacunas a endereçar
c)	Restabelecimento de acesso	*Não conforme*	Sem *backups* de `media/`; sem RTO/RPO formais.
d)	Avaliação regular da eficácia	Parcial	Sem SAST/DAST/SCA em *pipeline* CI; sem *pentest* externo formal.

Tabela 6: Sumário executivo da conformidade RGPD Art. 32.

A leitura honesta deste mapeamento é a seguinte: o ForensiQ tem posicionados os controlos correctos em CIA, integridade e auditoria, mas não está em condições de operar com dados reais sem que as lacunas das alíneas c) e d) sejam endereçadas. Esta posição é defensável em contexto académico e a sua identificação explícita é parte da contribuição do projecto.

2.5.3 Evidência externa de postura de segurança

Três serviços externos independentes auditaram a postura de segurança do ambiente de produção em forensiq.pt a 18 de Abril de 2026. Os relatórios completos estão em `docs/compliance/external-tests/`.

Teste	Serviço	Resultado
HSTS Preload Submission	hstspreload.org (Chromium / Mozilla / Edge / Apple)	Submetido para inclusão na lista pré-carregada
HTTP Observatory	Mozilla	Pontuação A+ (CSP, HSTS, Referrer-Policy, X-Content-Type-Options)
SSL Server Test	Qualys SSL Labs	Grau A+ (TLS 1.2/1.3, ECDHE, HSTS, OCSP *stapling*)

Tabela 7: Resultado dos três testes externos de segurança (18 Abr 2026).

A submissão à lista *HSTS preload* satisfaz o requisito funcional adicional de HTTPS estrito mesmo no primeiro acesso (sem janela de *trust on first use*), o que é particularmente relevante para um sistema que circula prova judicial.

2.6 Interface do utilizador

2.6.1 Princípios de desenho

A *interface* foi desenhada *mobile-first*, conforme ADR-0004, e respeita as Web Content Accessibility Guidelines 2.1 nível AA. As decisões foram orientadas pelo contexto operacional do agente: utilização com uma só mão, alvos tácteis de pelo menos 48×48 pixéis, contraste mínimo de 4,5:1 para texto não-decorativo, *focus rings* consistentes, `aria-busy` em listas dinâmicas, `aria-pressed` em *toggles* e *live regions* para anúncios. O sistema de cores inclui *tokens* semânticos por estado de custódia (`-state-apreendida`, `-state-em-pericia`, etc.).

A tipografia combina *Inter* para texto e *JetBrains Mono* para identificadores hexadecimais (*hashes*, IDs e *timestamps*). O modo claro e o modo escuro são alternáveis e activam-se automaticamente uma hora após o pôr do sol no local actual, calculado em *client-side* a partir da posição GPS e da fórmula NOAA (`settings.js:145-189`).

2.6.2 Vistas-chave

A aplicação organiza-se em torno de treze vistas principais, servidas por *templates* em `src/frontend/templates/`: autenticação (`/login/`); *dashboard* com saudação, acções rápidas e estatísticas por estado de custódia (`/dashboard/`); listagem e mapa de ocorrências (`/occurrences/`); criação de ocorrência por *wizard* (`/occurrences/new/`); *hub* da ocorrência (`/occurrences/<id>/`); listagem de evidências (`/evidences/`); *wizard* de evidência com captura de fotografia, GPS, *lookup* IMEI/VIN e sub-componentes (`/evidences/new/`); detalhe da evidência (`/evidences/<id>/`); *timeline* de custódia (`/evidences/<id>/custody/`); listagem de transições (`/custodies/`); estatísticas agregadas (`/stats/`); relatórios PDF (`/reports/`); definições de perfil e tema (`/settings/`).

2.6.3 Wizard de evidência

O *wizard* de criação de evidência é a *interface* principal de captura no terreno e está implementado em `src/frontend/templates/evidences_new.html:40-289` com oito passos sequenciais: (1) selecção da ocorrência; (2) tipo da evidência, com selector visual agrupado pelas duas grandes famílias (raízes e sub-componentes); (3) descrição livre; (4) data e hora de apreensão (preenchidas por omissão com o servidor); (5) detalhes específicos do tipo (*e.g.* IMEI para `MOBILE_DEVICE`, com botão de *lookup* externo); (6) fotografia (câmara nativa via atributo `capture="environment"` ou *upload* de ficheiro); (7) GPS (*Geolocation API* do *browser* + geocodificação inversa Nominatim em PT); (8) resumo com revisão completa antes da submissão.

2.6.4 Máquina de estados de custódia

A máquina de estados que governa a vida da prova é *linear* e sem retrocesso, com dois estados terminais. A Figura 6 apresenta o autómato; a sua implementação no dicionário `ChainOfCustody.VALID_TRANSITIONS` (`models.py:883-892`) foi já discutida em §2.3.5.

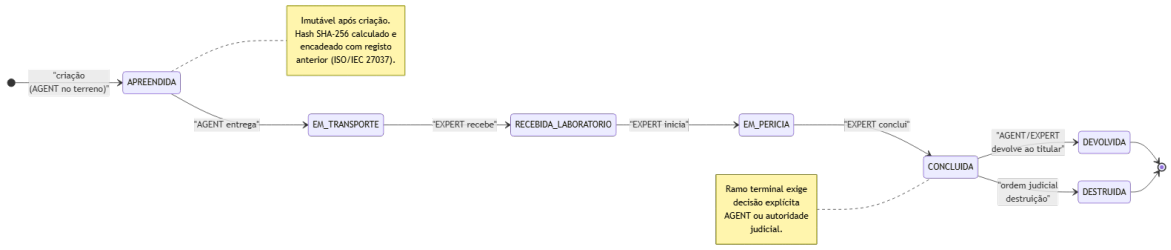

Figura 6: Máquina de estados da cadeia de custódia. Transições válidas a partir de cada estado; `DEVOLVIDA` e `DESTRUIDA` são terminais.

A transição inicial $\emptyset \rightarrow$ `APREENDIDA` é a única origem possível para uma evidência recém-registada; a partir daí o caminho canónico segue `APREENDIDA` → `EM_TRANSPORTE` → `RECEBIDA_LABORATORIO` → `EM_PERICIA` → `CONCLUIDA`, terminando em `DEVOLVIDA` ou `DESTRUIDA` consoante o destino judicial.

Limitação reconhecida — devolução directa por ordem do MP. A linearidade descrita acima é uma simplificação assumida da Fase 2 e cobre o caminho institucional padrão de uma apreensão que segue para perícia. O cenário operacional real, todavia, inclui um caso que esta máquina não acomoda: a *devolução directa por ordem do Ministério Público* — apreensão no terreno seguida, em prazo curto, de despacho do MP que ordena a entrega imediata do equipamento ao titular sem qualquer passagem por laboratório (tipicamente porque o tipo legal não justifica a perícia, ou porque a prova relevante já foi extraída por outra via). Na máquina actual, esse cenário obrigaria o agente a registar passos canónicos fictícios (`EM_TRANSPORTE`, `RECEBIDA_LABORATORIO`, `EM_PERICIA`, `CONCLUIDA`) só para alcançar `DEVOLVIDA`, o que é um anti-padrão de qualidade da prova: a cadeia ficaria documentalmente correcta mas factualmente falsa. A correcção planeada para o relatório final é cirúrgica: admitir as duas transições excepcionais `APREENDIDA` → `DEVOLVIDA` e `EM_TRANSPORTE` → `DEVOLVIDA` com requisito de *fundamento textual mínimo* no campo `observations` (tipicamente o NUIPC do despacho do MP que autoriza a entrega), mantendo o resto do autómato inalterado e a proibição de retrocesso intacta. A ramificação preserva a auditabilidade — cada devolução directa fica explicitamente referenciada a um despacho judicial — sem multiplicar a complexidade do modelo. Esta limitação é reconhecida aqui em cumprimento do princípio do *Guia de Projecto* segundo o qual fragilidades conhecidas e bem explicadas são preferíveis à sua ausência aparente (Pestana, 2026).

3 Implementação — Estado actual

Este capítulo documenta o estado de implementação à data deste relatório intercalar, fixado por *fecho técnico* no domingo 3 de Maio de 2026 e submetido a 6 de Maio (cumprindo o marco formal da UC), apresentando a decomposição de tarefas com precedências (§3.1), o estado por componente (§3.2), o cumprimento dos critérios de aceitação (§3.3), os desvios ao plano (§3.4) — incluindo o reconhecimento honesto da demo interna síncrona não realizada na janela prevista —, a demonstração em produção (§3.5), o estado dos testes automatizados (§3.6) e o calendário actualizado para a fase restante (§3.7).

3.1 Decomposição do trabalho

O trabalho foi decomposto em onze tarefas principais com precedências implícitas, conforme a Tabela 8.

T	Tarefa	Precedências	Estado
T1	Configuração do repositório, proposta e MoSCoW	—	Concluída

T	Tarefa	Precedências	Estado
T2	ADRs 0001–0010 com alternativas consideradas	T1	Concluída
T3	Modelo de dados core	T2	Concluída
T4	Autenticação JWT em *cookies* HttpOnly + CSRF	T3	Concluída
T5	API REST e *viewsets*	T3, T4	Concluída
T6	*Frontend vanilla mobile-first*	T5	Concluída
T7	Imutabilidade tripla (ORM, DRF, *triggers* PG)	T3, T5	Concluída
T8	Auditoria interna de segurança e correcções	T7	Concluída
T9	*Deployment* Fly.io + Neon; HSTS *preload*	T6	Concluída
T10	*Wizard* de evidência, *lookups*, exportação PDF	T5, T6	Concluída
T11	Relatório intercalar	T1–T10	Em curso

Tabela 8: Decomposição de tarefas com precedências.

3.2 Estado final por componente

A Tabela 9 resume o estado de cada componente. O estado verde indica conclusão validada por testes; amarelo indica funcionalidade entregue com limitação reconhecida; vermelho indica funcionalidade fora do âmbito do MVP ou ainda em curso.

Componente	Estado	Observação
Backend Django + DRF	verde	213 testes a passar; modelos, *viewsets*, serializadores, permissões.
Cadeia de custódia (SHA-256 encadeado)	verde	`transaction.atomic` + `select_for_update` (correcções da auditoria interna).
Imutabilidade tripla	verde	ORM + DRF + *triggers* PostgreSQL (*migration* 0002).
Autenticação JWT em *cookies*	verde	HttpOnly + Secure + SameSite=Strict; CSRF *double-submit*; *rate limit* 5/min.
Frontend vanilla	verde	13 vistas *mobile-first*; sem *build step*; tema dia/noite com sunset NOAA.
Mapa Leaflet + OpenStreetMap	verde	Leaflet 1.9.4 com SRI SHA-512; Referrer-Policy `strict-origin-when-cross-origin`.

Componente	Estado	Observação
Exportação PDF (RF6)	verde	ReportLab; `/api/evidences/<id>/pdf/` e `/api/occurrences/<id>/pdf/` com declaração ISO/IEC 27037.
Exportação CSV	verde	*Streaming* com *cap* de 10 000 linhas; respeita *ownership*; *audit log* regista cada exportação.
Lookup IMEI/VIN	verde	`DatabaseCache` 30 dias (IMEI) e 180 dias (VIN).
Deployment	verde	Fly.io fra + Neon; HTTPS A+ (Qualys); HSTS *preload* submetido; Mozilla Observatory A+.
Cobertura de testes	amarelo	71,5% global (1881 *statements*, 536 não cobertos); áreas com cobertura insuficiente reconhecidas (`validators.py`, `imei_lookup.py`, `seed_demo.py`).
Backups de `media/`	vermelho	Volume Fly.io persistente sem *backup* externo automatizado; trabalho futuro.
SAST/SCA/DAST em CI	vermelho	Fora do âmbito do MVP; documentado para Fase 3.

Tabela 9: Estado por componente à data do intercalar.

3.3 Cumprimento dos critérios de aceitação

A Tabela 10 confronta o estado dos requisitos funcionais (RF) e não-funcionais (RNF) inscritos em `docs/scope/requirements.tex` (v1.0) com a sua verificação empírica à data deste intercalar. Os requisitos *Must have* foram todos cumpridos. Os *Should have* foram cumpridos. Os *Could have* estão parcialmente cumpridos. Os *Won't have* permaneceram fora do âmbito.

ID	Requisito	Prioridade	Estado
RF01	Autenticação JWT com perfis	Must	Cumprido
RF02	Ocorrências com georreferenciação	Must	Cumprido
RF03	Registo de prova com foto, GPS, *hash*	Must	Cumprido
RF04	Ficha de dispositivo digital	Must	Cumprido
RF05	Cadeia de custódia *append-only* encadeada	Must	Cumprido
RF06	Exportação PDF	Must	Cumprido
RF07	API REST com 10+ *endpoints* documentados	Must	Cumprido (mais de 12 *endpoints*)
RF08	*Dashboard* por estado	Should	Cumprido
RF09	Pesquisa e filtragem	Should	Cumprido

ID	Requisito	Prioridade	Estado
RF10	Perfis adicionais (coordenador, magistrado)	Could	Estrutura preparada (ADR-0006)
RF11	Verificação de IMEI via *API* externa	Could	Cumprido (ADR-0008)
RF12	*Dashboard* analítico	Could	Cumprido (`/stats/`)
RF13	Despachos judiciais	Could	Não realizado
RF14–RF17	Integração PSP, dados reais, módulos QBF, *app* nativa	Won't	Fora do âmbito
RNF01	HTTPS obrigatório	Must	Cumprido (HSTS *preload* + Qualys A+)
RNF02	*Mobile-first* utilizável com uma só mão	Must	Cumprido (WCAG 2.1 AA)
RNF03	Integridade SHA-256 e *append-only*	Must	Cumprido
RNF04	Resposta inferior a 2 s	Should	Verificado em produção
RNF05	Testes automatizados em módulos críticos	Should	213 testes a passar
RNF06	*Deployment* público	Could	forensiq.pt

Tabela 10: Cumprimento dos requisitos funcionais e não-funcionais.

3.4 Desvios ao plano

A execução afastou-se em três pontos do plano apresentado na proposta; todos foram comunicados ao orientador.

***Stack* do *backend*.** A proposta indicava *"Django + DRF"* como opção principal, sem alternativa. A escolha foi confirmada em ADR-0002 e mantida. Sem desvio.

Demo interna síncrona não realizada na janela prevista (Sem. 7). O calendário do orientador previa demo interna síncrona via Teams entre 28 de Abril e 2 de Maio de 2026 (Sem. 7). A demo síncrona *não foi realizada* nessa janela. O autor reconhece o desvio. Como mitigação, propõe-se ao orientador, em comunicação paralela a este relatório, que a demonstração seja feita de forma *assíncrona* através do site em produção forensiq.pt, com credenciais de demonstração entregues por canal separado. Esta modalidade permite ao orientador exercitar todos os fluxos críticos (criação de ocorrência, registo de evidência com GPS e fotografia, transição de custódia, exportação de PDF) sem necessidade de configurar ambiente local. Caso o orientador prefira demo síncrona, o autor está disponível para a agendar nas semanas 9 ou 10 (7–16 mai), com a vantagem de já ter incorporado o feedback do intercalar.

Auditorias internas extras (16, 18 e 19 Abr). O plano original avançava da implementação directamente para o relatório intercalar. Em meados de Abril realizaram-se três auditorias internas (segurança, *design*, taxonomia) que detectaram pontos a corrigir, em particular uma *race condition* na cadeia de custódia (*fix B-C3*) e a necessidade de tornar o cálculo do *hash* puro (*fix B-C2*), ambos documentados em `docs/AUDIT_2026-04-19.md`.

Em revisão final pré-intercalar foram ainda detectados e corrigidos: a ausência de *triggers* de imutabilidade em `core_occurrence` (*fix* resolvido em *migration* `0013`, ver §2.3.6) e o caso degenerado de concorrência no primeiro registo de cadeia de custódia (*fix B-C4*, lock adicional na `Evidence` parent, ver §2.3.5). O esforço adicional consumiu cerca de uma semana mas foi essencial para a robustez actual.

Operação *offline* no terreno — âmbito ajustado. A Necessidade 3 (utilidade no terreno, §1.3) foi originalmente formulada com a expressão *redes móveis variáveis*, que a leitura crítica posterior revelou ambígua. O MVP exige ligação activa para abrir o *wizard* de evidência e submeter o registo: não há *Service Worker* nem fila de submissão local em `IndexedDB`. A operação em ausência total de cobertura (cave, garagem, parque subterrâneo) é, portanto, indisponível na versão actual. O âmbito da Fase 2 fica delimitado a *ligação móvel intermitente com janela de cobertura disponível no momento da submissão*; a robustez offline com PWA leve (*cache* de *assets* e fila de submissões assinadas) está identificada como trabalho futuro de Fase 3 e como prioridade alta na evolução pós-licenciatura.

Fragilidades de modelo identificadas na revisão pré-intercalar. A revisão crítica conduzida nas semanas finais antes do fecho deste relatório identificou duas fragilidades de modelo que ficam reconhecidas nas secções técnicas correspondentes (§2.6.4 e §2.4.3) e cuja correcção em código está calendarizada para a iteração de 7–16 mai, antes do relatório final: *(i)* ramificação excepcional da máquina de estados de custódia para acomodar a devolução directa por ordem do Ministério Público sem passagem por laboratório; *(ii)* adição da directiva `style-src-attr 'unsafe-inline'` à política de segurança de conteúdos, resolvendo o conflito CSP nível 3 com o Leaflet sem relaxar o restante da postura. Ambas as alterações são confinadas a entradas pontuais (um dicionário de transições; uma directiva adicional) e não exigem revisão arquitectural.

3.5 Demonstração em produção

O ambiente de produção em forensiq.pt é uma cópia plenamente funcional do ForensiQ, alojada em Fly.io Frankfurt com PostgreSQL gerido pela Neon na mesma região (§2.2.2). Está disponível um conjunto de demonstração realista, gerado pelo *management command* `seed_demo` (`src/backend/core/management/commands/seed_demo.py`), com cinco ocorrências em cidades portuguesas (Lisboa, Porto, Coimbra, Braga, Faro), respectivos NUIPCs e doze itens de prova distribuídos pelos vários tipos da taxonomia, incluindo cartões SIM como sub-componentes hierárquicos (a contagem por ocorrência é 2–3–1–2–4). As fotografias são imagens *placeholder* sintéticas geradas pelo *seed* para evitar qualquer dado pessoal real.

O acesso à demonstração é feito com três contas geradas no momento do *seed* com *passwords* aleatórias (16 caracteres): `agente.demo` (perfil `AGENT`, *badge* `PSP-12345`), `perito.demo` (perfil `EXPERT`, *badge* `PJ-LPC-007`) e `pedro.pestana` (perfil `EXPERT` com *flag* `is_staff`, *badge* `UAB-ORIENT`). As *passwords* são entregues ao orientador em comunicação paralela ao envio deste relatório, e devem ser rodadas por ele ao primeiro acesso.

3.6 Estado dos testes automatizados

À data deste intercalar, a *suite* automatizada do ForensiQ contém **213 testes a passar** sob `pytest 9.0.3` (execução em `2026-05-03`, tempo total `0:02:47`). A distribuição por *suite* é: `tests.py` (14 testes de modelos), `tests_api.py` (83 testes de API: autenticação, IDOR, imutabilidade, transições, *throttle*, *lookups*, estatísticas, CSP, *rate limiting*), `tests_pdf.py` (18 testes de exportação PDF), `tests_frontend.py` (54 testes de vistas e *templates*), `tests_new_features.py` (22 testes de funcionalidades introduzidas em iterações posteriores) e `tests_table_mode.py` (22 testes de modo tabela e exportação CSV).

A cobertura medida por `coverage.py` é de **71,5%** global em `core/` ($1881 - 536 = 1345$ *statements* cobertos em 1881). A leitura honesta destes números é a seguinte: a cobertura concentra-se nas áreas críticas para integridade forense — `models.py`, `views.py`, `pdf_export.py` e `middleware.py`, todas acima de 75%; a cobertura global é puxada para baixo por três módulos: `seed_demo.py` (*script* de demonstração executado manualmente, intencionalmente sem testes), `imei_lookup.py` (integração externa que exigiria *mocking* mais sofisticado) e `validators.py` (cujos casos felizes estão cobertos indirectamente pelos testes de criação de evidência). O reforço da cobertura para $\geq 80\%$ é uma das prioridades das semanas 9–10 (§3.7).

3.7 Calendário actualizado para a fase restante

A Tabela 11 apresenta o calendário actualizado para a fase restante, da semana seguinte ao intercalar até à entrega do relatório final em 24 de Junho de 2026.

Sem.	Datas	Trabalho previsto
9–10	7–16 mai	Reforço de cobertura de testes (alvo $\geq 75\%$); *property-based testing* de validadores; eventual demo síncrona com o orientador.
11–12	19–30 mai	Polimento da *interface mobile-first*; tabela de auditoria visual no *dashboard*; preparação dos anexos do relatório final (manifesto de testes, capturas de ecrã).
13	2–6 jun	Revisão geral; validação cruzada dos critérios de aceitação; congelamento do âmbito.
14	9–13 jun	Redacção dos Capítulos 4 (Testes) e 5 (Conclusões) do relatório final; bibliografia APA consolidada.
15	16–20 jun	Reunião de preparação para defesa com o orientador (síncrono); ensaio de perguntas de júri.
16	24 jun	Submissão do relatório final e do código.

Tabela 11: Calendário actualizado para a fase restante.

Uso de IA generativa

Em cumprimento da secção 8 do *Guia de Projecto* do orientador (Pestana, 2026), declara-se a seguir o uso de ferramentas de *IA generativa* no desenvolvimento do ForensiQ. O critério aplicado foi o do guia: utilização aceite desde que o autor compreenda e seja capaz de defender, perante o júri, todo o conteúdo produzido.

Ferramenta principal. **Claude** (Anthropic), modelo *Opus 4.x* acedido via *Claude Code* CLI, foi usado em modo *pair programming* ao longo do semestre, em particular para:

- geração inicial de *boilerplate* (*models*, *serializers*, *viewsets* DRF) revisto e adaptado pelo autor antes de ser aceite;
- exploração de alternativas de arquitectura (incorporadas nos ADRs com decisão final do autor);
- revisão crítica de segurança, com aplicação manual de cada correcção (auditorias internas de 16, 18 e 19 Abr 2026);
- geração de testes a partir de especificações do autor (a interpretação dos *edge cases* foi sempre validada manualmente);
- revisão de redacção, ortografia e coerência terminológica do presente relatório.

Disciplina aplicada. Toda a saída de IA foi compreendida, verificada e adaptada antes de entrar no repositório. Decisões de arquitectura registadas nos dez ADRs em `docs/architecture/adr/` foram tomadas pelo autor; a IA serviu de interlocutor para testar essas decisões, não para as substituir. O uso de IA **não** se estendeu a: tomada de decisões de âmbito, interpretação de requisitos do orientador, validação de critérios de aceitação, ou redacção de qualquer afirmação técnica que o autor não consiga defender em sede de defesa pública.

Outras ferramentas. Para além do Claude foram usados, pontualmente, o *GitHub Copilot* (sugestões *inline* em ficheiros isolados) e *ChatGPT* (consulta de exemplos de *packages* LaTeX). Não foi usada IA na geração das credenciais *seed*, na assinatura criptográfica de qualquer registo, nem na auditoria externa (Qualys SSL Labs, Mozilla Observatory, HSTS Preload), que são serviços independentes auditáveis.

Referências

Association of Chief Police Officers (2012). Good practice guide for digital evidence. Technical report, ACPO. Version 5.

Brown, S. (2014). *Software Architecture for Developers*. Leanpub.

Casey, E. (2011). *Digital Evidence and Computer Crime: Forensic Science, Computers and the Internet*. Academic Press, 3 edition.

ISO/IEC (2012). Information technology — security techniques — guidelines for identification,

collection, acquisition and preservation of digital evidence. ISO/IEC 27037:2012. International Organization for Standardization.

Parlamento Europeu e Conselho da União Europeia (2016). Regulamento (ue) 2016/679 relativo à protecção das pessoas singulares no que diz respeito ao tratamento de dados pessoais e à livre circulação desses dados. Jornal Oficial da União Europeia, L 119.

Pestana, P. D. (2025). Plataforma modular de captura e preservação de evidência digital para OSINT. Projecto #38 — LEI 2025/26, Universidade Aberta.

Pestana, P. D. (2026). Guia de projecto de engenharia informática — 2025/26. Versão 4.0, Universidade Aberta.

Sage, D. (2024). Django REST framework simple JWT. https://django-rest-framework-simplejwt.readthedocs.io/.